



Secure Software Security

- Lecture By
Pratidnya S. Hegde Patil
Assistant Professor-IT Dept.



Topics

- Impact of security on the various phases of the software lifecycle.
- Secure Software Lifecycle:
 - Need (CYBOK-16.1),
 - SecSDL (Microsoft SDL, Touchpoints , SAFECode (16.2),
 - Adaption of SecSDL to agile, mobile and cloud environment(16.3.1-16.3.3),
 - Assessing the Secure software lifecycle (SAMM, BSIMM, Common Criteria) (16.4).



History (Reactive Approach)

- Security is assessed when “complete” or “deployed”; discover vulnerabilities then fixed/patched
 - Costly
 - Attackers can find and exploit vulnerabilities without being noticed
 - (Urgent) Patches can introduce new vulnerabilities
 - Patches go unapplied by customers



Shortcomings with Historic Reactive Approach

- **Breaches are costly:** Loss of reputation caused by a breach is difficult to quantify.
- **Attackers can find and exploit vulnerabilities without being noticed:** Once a vulnerability is publicly reported, attackers formulate a new mechanism to exploit the vulnerability with the knowledge that many organisations will not adopt the fix.
- **Patches can introduce new vulnerabilities or other problems:** Vulnerability patches are considered urgent and can be rushed out, potentially introducing new problems to a system.
- **Patches often go unapplied by customers:** Users and system administrators may be reluctant to apply security patches.



Proactive Approach

- Proactive rigorous software analysis should play an increasingly important role in assessing and preventing vulnerabilities in applications based upon the well-known fact that security violations occur because of errors in software design and coding.
- Building Secure Software in 2002, with a focus on preventing the injection of vulnerabilities and reducing security risk through an integration of security into a software development process.



Purpose of Secure Software Lifecycle

- The purpose of this Secure Software Lifecycle knowledge area is to provide an overview of software development processes for implementing **secure software** from the design of the software to the operational use of the software.
- This implementation may involve **new coding** as well as the incorporation of **third-party libraries and components**.



Need for Secure Software Lifecycle

- Important for an organization to have a comprehensive secure software lifecycle.
- Organizations usually customize their own software lifecycle (rather than take a prescriptive approach)
- Successful adoption requires cultural change (in addition to adopting the technical practices)



Security Software Lifecycle

- The components of a comprehensive software development process to prevent and detect all classes of security defects and to respond in the event of an exploit.



Secure Software Lifecycle Processes

- Microsoft Security Development Lifecycle (SDL)-(12 practices)
- Touchpoints-(7 practices)
- SAFECODE-(8 practices)



Microsoft Security Development Lifecycle (SDL)-(12 practices)

1. Providing Training :

Software developers and architects must understand technical approaches for preventing and detecting vulnerabilities. The entire development organisation should be cognisant of the attacker's perspective, goals, and techniques; and of the business implications of not building secure products. Assess organizational knowledge on security and privacy – establish training program as necessary

- Establish training criteria
 - Content covering secure design, development, test and privacy
 - (ex: 80% of all technical personnel trained)
- Establish minimum training frequency – attackers are a moving target
 - Employees must attend n classes per year
- Need to cover all the topics in the SDL



Microsoft Security Development Lifecycle (SDL)-(12 practices)

2. Security Requirements :

Consider security at the outset of a project. Security requirements should be defined during the initial design and planning phases. Factors that influence these requirements include the specific functional requirements of the system, the legal and industry compliance requirements, internal and external standards, previous security incidents, and known threats. Security requirements must be continually updated to reflect changes in required functionality, standards, and the threat landscape.

- Development team identifies security requirements.
- Factor in security implications of functional requirements, legal and industry, compliance, standards, known threats, previous security incidents.
- Techniques for systematically developing examples: SQUARE, abuse cases, i* and KAOS

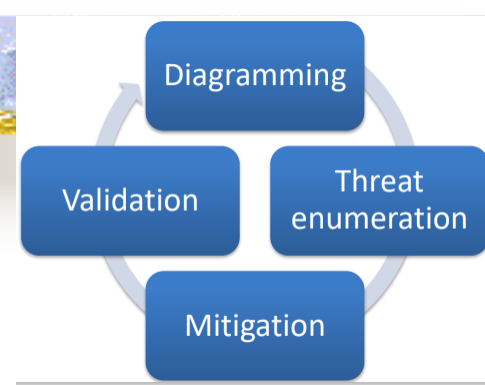


Microsoft Security Development Lifecycle (SDL)-(12 practices)

3. Define Metrics and Compliance Reporting :

- Define and document a bug bar for security.
 - Classification of what are moderate, important, or critical security and privacy bug types
 - User to set priority for fixing and to determine if the product can ship
- Ensure that bug reporting tools can track security and issues and that a database can be queried dynamically for all security bugs
- Understand requirement compliance and associate reporting requirements.

Microsoft Security Development Lifecycle (SDL)- (12 practices)



4. Perform Threat Modelling :

- Threat modelling is a process to understand (potential) security threats to a system, determine risks from those threats, and establish appropriate mitigations.
- An approach is to consider the (1) the malicious and benevolent interactors with the system; (2) the design of the system and its components (i.e. processes and data stores), (3) the trust boundaries of the system; and (4) the data flow of the system within and across trust boundaries to/from its interactors.



4. Perform Threat Modelling :

- Threats can be enumerated using a systematic approach of considering each system component relative to the STRIDE (Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege) threats:
 - **Spoofing identity.** Spoofing threats allow an attacker to pose as another user or allow a rogue server to pose as a valid server.
 - **Tampering with data.** Data tampering threats involves malicious modification of data.
 - **Repudiation.** Repudiation threats are associated with users who deny performing an action without other parties having any way to prove otherwise.
 - **Information disclosure.** Information disclosure threats involve the exposure of information to individuals who are not supposed to have access to it.
 - **Denial of service.** A Denial of Service (DoS) attack denies service to valid users by making the system unavailable or unusable.



Microsoft Security Development Lifecycle (SDL)-(12 practices)

5. Establish design requirements :

- Consider design principles, such as:
 - Economy of mechanism: Keep the design of the system as simple and small as possible.
 - Fail-safe defaults: Design a security mechanism so that a failure will follow the same execution path as disallowing the operation.
 - Complete mediation: Design a security mechanism so that a failure will follow the same execution path as disallowing the operation.
 - Open Design:: The design should not depend upon the ignorance of attackers but rather on the possession of keys or passwords.
 - Separation of privilege: A protection mechanism that requires two keys to unlock is more robust than one that requires a single key when two or more decisions must be made before access should be granted.



Microsoft Security Development Lifecycle (SDL)-(12 practices)

5. Establish design requirements :

- Consider design principles, such as:
 - Least privilege: Every program and every user should operate using the least set of privileges necessary to complete the job.
 - Least common mechanism: Minimise the amount of mechanisms common to more than one user and depended on by all users.
 - Psychological acceptability: The human interface should be designed for ease of use so that users routinely and automatically apply the mechanisms correctly and securely.
 - Defense in depth: Provide multiple layers of security controls to provide redundancy in the event a security breach.
 - Design for updating: The software security must be designed for change, such as for security patches and security property changes.

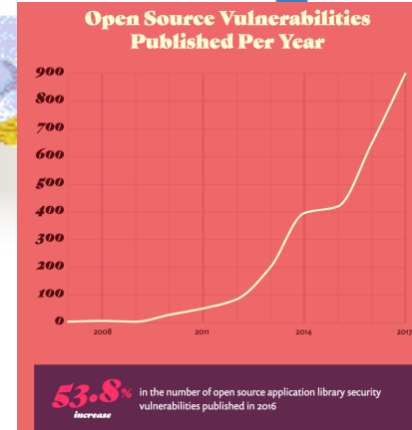


Microsoft Security Development Lifecycle (SDL)-(12 practices)

6. Define and Use Cryptography Standards :

- Through the proper use of cryptography, one can protect the confidentiality of data, protect data from unauthorized modification, and authenticate the source of data.

Microsoft Security Development Lifecycle (SDL)-(12 practices)



7. Manage the security risk of using 3rd party components :

An organisation should have an accurate inventory of third-party components, continuously use a tool to scan for vulnerabilities in its components, and have a plan to respond when new vulnerabilities are discovered. <https://snyk.io/stateofossecurty/>



Microsoft Security Development Lifecycle (SDL)-(12 practices)

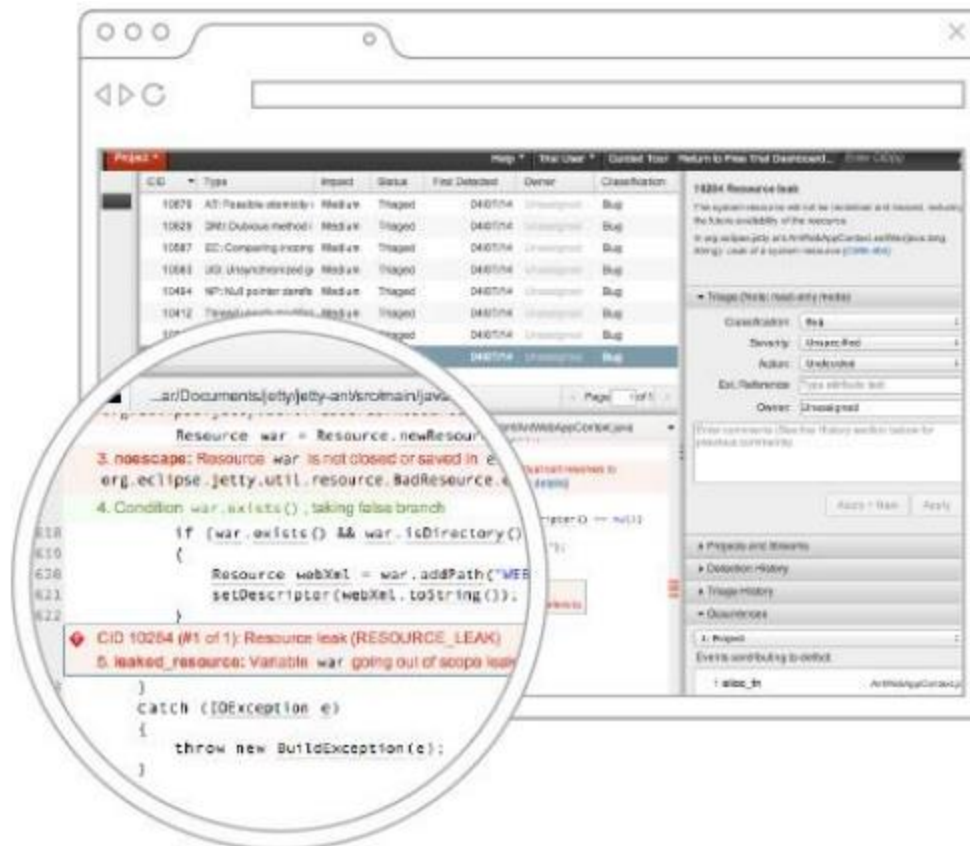
8. Use approved tools :

- The resultant software must be backward compatible with previous versions.
- Specification of approved build tools and options
 - Compilers and code generators
 - Static analysis
 - Debuggers
 - Dynamic analysis
 - Test verification
 - IDE
- Decide on settings and ensure settings are correct
- Security isn't static
 - Update your tools because they change with dynamic threat environment

http://www.ultraedit.com/support/tutorials_power_tips/uestudio/integerated_debugger.html

Microsoft Security Development Lifecycle (SDL)-(12 practices)

9. Perform static analysis security testing (SAST)





Microsoft Security Development Lifecycle (SDL)-(12 practices)

10. Perform dynamic analysis security testing (DAST) :

DAST performs run-time verification of compiled or packaged software to check functionality that is only apparent when all components are integrated and running. DAST often involves the use of a suite of pre-built attacks and malformed strings that can detect memory corruption, user privilege issues, injection attacks, and other critical security problems.



Microsoft Security Development Lifecycle (SDL)-(12 practices)

11. Perform penetration testing :

Manual pentesting is black box testing of a running system to simulate the actions of an attacker. Pentesting is often performed by skilled security professionals, who can be internal to an organisation or consultants, opportunistically simulating the actions of a hacker. The objective of a penetration test is to uncover any form of vulnerability - from small implementation bugs to major design flaws resulting from coding errors, system configuration faults, design flaws or other operational deployment weaknesses.



Microsoft Security Development Lifecycle (SDL)-(12 practices)

12. Establish a standard incident response practice :

Creation of a clearly defined support policy

- Prepare Cyber Security Incident Response Plan (CSIRP)
 - Identify contact for Cyber Security Council and resources to respond to incidents
 - 24x7x365 contact information for engineering, marketing and management individuals with decision-making authority
 - Ensure ability to service all code “emergency” releases and all licensed 3rd party code



Adaptations of the Secure Software Lifecycle in various domains

- Agile Software Development and DevOps
- Mobile
- Cloud Computing
- Internet of Things (IoT)
- Road Vehicles
- eCommerce/Payment card Industry



Adaptations

■ Agile

- Agile and continuous software development methodologies are highly iterative, with new functionality being provided to customers frequently- potentially as quickly as multiple times per day or as 'slowly' as every two to four weeks. Agile software development methodologies can be functional requirement-centric, with the functionality being expressed as user stories.
- SAFECode provides security user stories
- With a DevOps approach to developing software, development and operations are tightly integrated to enable fast and continuous delivery of value to end users..
- Microsoft Secure DevOps
 - Provide Training
 - Define Requirements
 - Define Metrics and Compliance Reporting
 - Use Software Composition Analysis (SCA)
 - Perform Threat Modelling
 - Use Tools and Automation
 - Keeping credentials safe
 - Continuous learning and monitoring



Adaptations

■ Mobile

Security concerns for mobile apps differ from traditional desktop software in some important ways, including local data storage, inter-app communication, proper usage of cryptographic APIs and secure network communication. The OWASP Mobile Security Project is a resource for developers and security teams to build and maintain secure mobile applications.

Four resources are provided to aid in the secure software lifecycle of mobile applications:

- ❑ OWASP Mobile Application Security Verification Standard (MASVS) Security Requirements and Verification
- ❑ Mobile Security Testing Guide (MSTG)
- ❑ Mobile App Security Checklist
- ❑ Mobile Threat Model



Adaptations

■ Cloud Computing

Security concerns for mobile apps differ from traditional desktop software in some important ways, including local data storage, inter-app communication, proper usage of cryptographic APIs and secure network communication. The OWASP Mobile Security Project is a resource for developers and security teams to build and maintain secure mobile applications.

These threats and associated practices are:

- ❑ Threat: Multitenancy
- ❑ Tokenisation of Sensitive data
- ❑ Trusted Compute Pools
- ❑ Data Encryption and Key Management
- ❑ Authentication and Identity Management
- ❑ Shared-Domain Issues



Adaptations

■ Internet of Things (IoT)

- Devices must be securely provisioned, connectivity between these devices and the cloud must be secure, and data in storage and in transit must be protected. The US National Institute of Standards and Technology (NIST) recommends four practices for the development of secure IoT-based systems:
 1. Use of Radio-Frequency Identification (RFID) tags: Unique identifiers can mitigate the problem of counterfeits, data tampered, deleted, dropped or transmitted insecurely. Readers activate a tag, causing the device to broadcast radio waves within a bandwidth reserved for RFID usage by governments internationally. The radio waves transmit identifiers or codes that reference unique information associated with the device.



Adaptations

■ Internet of Things (IoT)

2. Not using or allowing the use of default passwords or credentials: Recommended practices are to require passwords to be changed or to design in intuitive interfaces. Manufacturers can randomise passwords per device rather than having a small number of default passwords.
3. Use of the Manufacturer Usage Description (MUD) specification: MUD allows manufacturers to specify authorised and expected user traffic patterns to reduce the threat surface of an IoT device by restricting communications to/from the device to sources and destinations intended by the manufacturer.



Adaptations

■ Internet of Things (IoT)

4. Development of a Secure Upgrade Process: In non-IT systems, updates are usually delivered via a secure process in which the computer can authenticate the source pushing the patches and feature and configuration updates. The IoT Firmware Update Architecture provides guidance on implementing a secure firmware update architecture including hard rules defining how device manufacturers should operate.



Adaptations

■ Road Vehicles

- The team should follow a secure product development process based on a systems-engineering approach with the goal of designing systems free of unreasonable safety risks including those from potential cyber security threats and vulnerabilities.
- The automotive industry should have a documented process for responding to incidents, vulnerabilities and exploits. This process should cover impact assessment, containment, recovery and remediation actions, the associated testing, and should include the creation of roles and responsibilities for doing so. The industry should also establish metrics to periodically assess the effectiveness of their response process.



Adaptations

■ Road Vehicles

- The automotive industry should document the details related to their cyber security process, including the results of risk assessment, pen testing and organisations decisions related to cyber security. Essential documents, such as cyber security requirements, should follow a robust version control protocol.
- These security requirements should be incorporated into the product's security requirements:
 - Limit developer/debugging access to production devices, such as through an open debugging port or through a serial console.
 - Keys (e.g., cryptographic) and passwords which can provide an unauthorised, elevated level of access to vehicle computing platforms should be protected from disclosure. Keys should not provide access to multiple vehicles.



Adaptations

■ Road Vehicles

- These security requirements should be incorporated into the product's security requirements:
 - Diagnostic features should be limited to a specific mode of vehicle operation which accomplishes the intended purpose of the associated feature. For example, a diagnostic operation which may disable a vehicle's individual brakes could be restricted to operating only at low speeds or not disabling all the brakes at the same time.
 - Encryption should be considered as a useful tool in preventing the unauthorised recovery and analysis of firmware.
 - Limit the ability to modify firmware and/or employ signing techniques to make it more challenging for malware to be installed on vehicles.
 - The use of network servers on vehicle ECUs should be limited to essential functionality, and services over these ports should be protected to prevent use by unauthorised parties.
 - Logical and physical isolation techniques should be used to separate processors, vehicle networks, and external connections as appropriate to limit and control pathways from external threat vectors to cyber-physical features of vehicles.



Adaptations

■ Road Vehicles

- These security requirements should be incorporated into the product's security requirements:
 - Sending safety signals as messages on common data buses should be avoided, but when used should employ a message authentication scheme to limit the possibility of message spoofing.
 - An immutable log of events sufficient to enable forensic analysis should be maintained and periodically scrutinised by qualified maintenance personnel to detect trends of cyber-attack.
 - Encryption methods should be employed in any IP-based operational communication between external servers and the vehicle, and should not accept invalid certificates.
 - Plan for and design-in features that could allow for changes in network routing rules to be quickly propagated and applied to one, a subset or all vehicles



Adaptations

- eCommerce / Payment Card Industry (PCI)

The ability to steal large quantities of money makes the PCI an attractive target for attackers. PCI created the Security Standards Council, a global forum for the ongoing development, enhancement, storage, dissemination, and implementation of security standards for account data protection.

- Data Security Standard (PCI DSS)

- PCI DSS contains 12 requirements that businesses are required to implement to protect credit card data:

1. Install and maintain a firewall configuration to protect cardholder data.
2. Do not use vendor-supplied defaults for system passwords and other security parameters.
3. Protect stored cardholder data.
4. Encrypt transmission of cardholder data across open, public networks.
5. Use and regularly update antivirus software.
6. Develop and maintain secure systems and applications, including detecting and mitigating vulnerabilities and applying mitigating controls.



Adaptations

- eCommerce / Payment Card Industry (PCI)
 - PCI DSS contains 12 requirements that businesses are required to implement to protect credit card data:
 7. Restrict access to cardholder data by business need-to-know.
 8. Assign a unique ID to each person with computer access.
 9. Restrict physical access to cardholder data.
 10. Track and monitor all access to network resources and cardholder data.
 11. Regularly test security systems and processes.
 12. Maintain a policy that addresses information security.

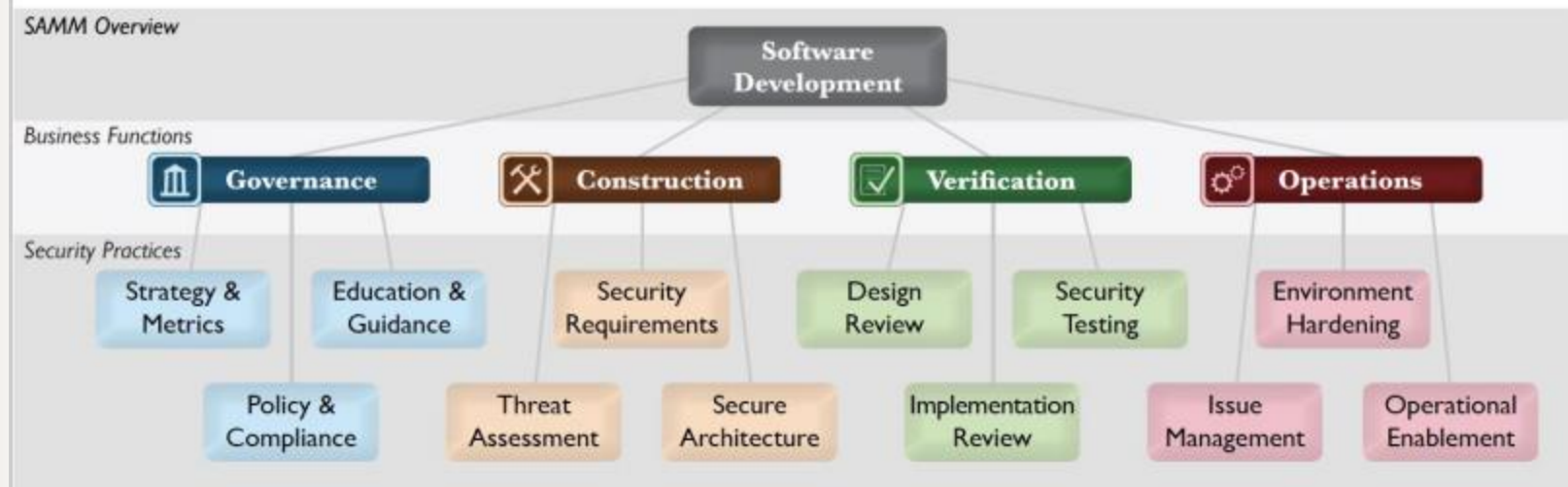


Frameworks to Assess the Secure Software Lifecycle

- SAMM
- BSIMM
- Common Criteria

Assessing the Secure Software Lifecycle

Open SAMM Framework





SAMM

- The Software Assurance Maturity Model (SAMM) is an open framework to help organisations formulate and implement a strategy for software security that is tailored to the specific risks facing the organisation. Resources are provided for the SAMM to enable an organisation to do the following:
 1. Define and measure security-related activities within an organisation.
 2. Evaluate their existing software security practices.
 3. Build a balanced software security program in well-defined iterations.
 4. Demonstrate improvements in a security assurance program.
- The business functions and security practices are as follows:
 1. Business Function: Governance
 - (a) Strategy and metrics
 - (b) Policy and compliance
 - (c) Education and guidance



SAMM

■ The business functions and security practices are as follows:

2. Business Function: Construction

- (a) Threat assessment
- (b) Security requirements
- (c) Secure architecture

3. Business Function: Verification

- (a) Design review
- (b) Code review
- © Security testing

4. Business Function: Deployment

- (a) Vulnerability management
- (b) Environment hardening
- © Operational enablement

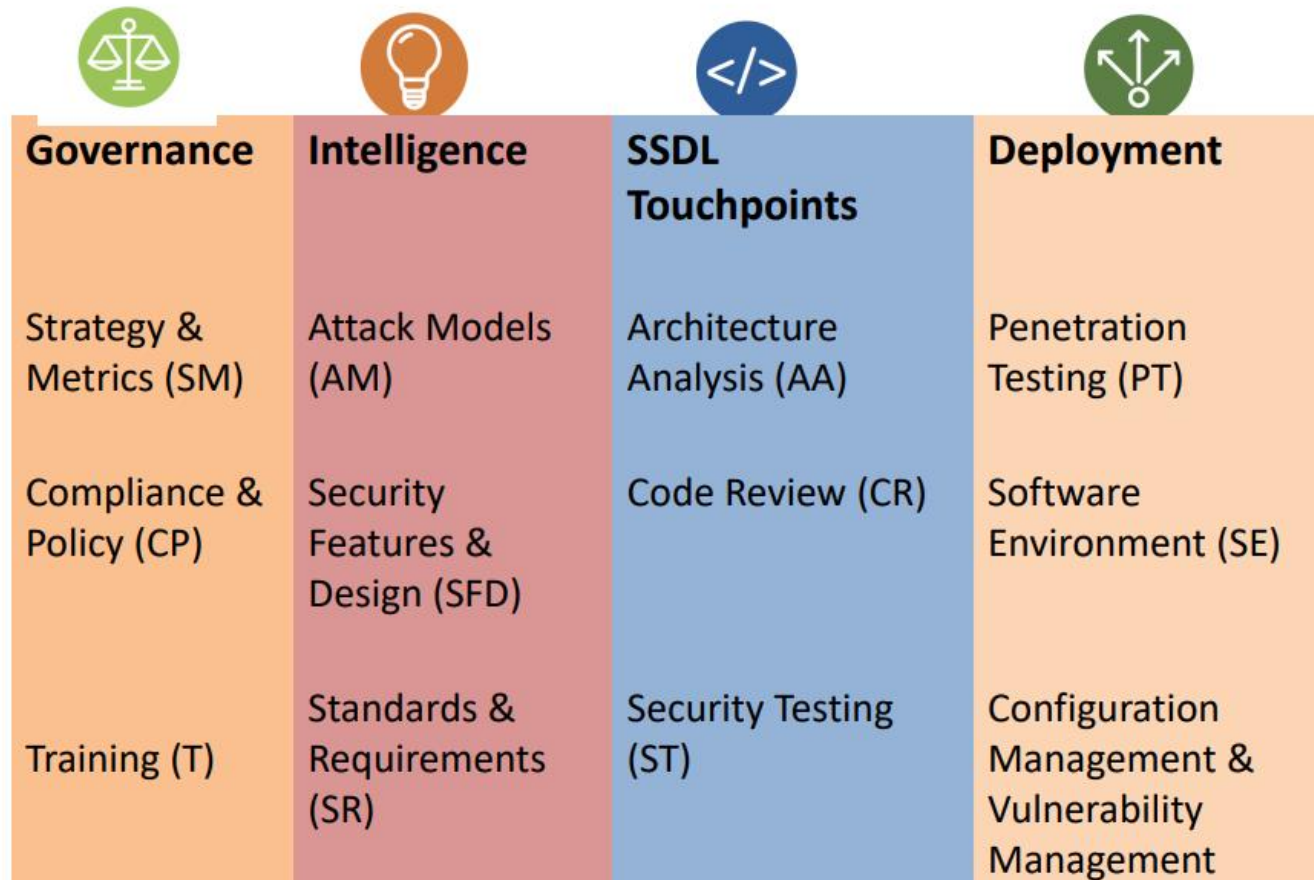


SAMM

- SAMM assessments are conducted through self-assessments or by a consultant chosen by the organisation. Spreadsheets are provided by SAMM for scoring the assessment, providing information for the organisation on their current maturity level:
 - 0: Implicit starting point representing the activities in the Practice being unfulfilled.
 - 1: Initial understanding and ad hoc provision of the Security Practice.
 - 2: Increase efficiency and/or effectiveness of the Security Practice.
 - 3: Comprehensive mastery of the Security Practice at scale.
- Assessments may be conducted periodically to measure improvements in an organisation's security assurance program

Assessing the Secure Software Lifecycle

BSIMM Framework





BSIMM

The domains and practices are as follows:

1. Domain: Governance

- (a) Strategy and metrics
- (b) Compliance and policy
- (c) Training

2. Domain: Intelligence

- (a) Attack models
- (b) Security features and design
- (c) Standards and requirements

3. Domain: Secure software development lifecycle touchpoints

- (a) Architecture analysis
- (b) Code review
- (c) Security testing

4. Domain: Deployment

- (a) Penetration testing
- (b) Software environment
- (c) Configuration management and vulnerability management



Objectives of the Common Criteria

- Permits comparability between the results of independent security evaluations by providing common set of requirements for secure functionality of IT products
- Establishes level of confidence in the security functionality of IT products
- May help consumers determine whether the IT product fulfils their security needs